

ASSIGNMENT 3

Course: Machine Learning for Robotics

Topic: Logistic Regression

Total Marks: 100

Submission Type: Individual

Instructions

1. You are allowed to use ONLY:
 - numpy
 - matplotlib
 - pandas (only for loading CSV files)
 - sklearn.datasets.make_blobs (ONLY for dataset generation)
2. You are NOT allowed to use:
 - sklearn.linear_model
 - sklearn.metrics
 - scipy.optimize
 - torch
 - tensorflow
 - keras
 - any built-in optimizer
 - any pre-written logistic regression implementation

Failure to follow instructions will result in zero marks.

Submission Requirements

Submit a Jupyter Notebook (.ipynb) with all your code.

Objective:

Implement Binary Logistic Regression from scratch and visualize the decision boundary.

Part 1 : Dataset Creation (10 Marks)

- Generate a 2D binary classification dataset using: `from sklearn.datasets import make_blobs`
- Conditions:
 - Exactly 2 features (each sample is a 2D point `(x1, x2)`)
 - Exactly 2 classes (0 or 1)
 - At least 300 samples
 - Set `random_state` = first two digits of your roll number (e.g., 22L-4202 → 42)
- Plot the dataset clearly:
 - Label axes
 - Add legend
 - Show class separation

Part 2 : Function Implementation (25 Marks)

- You are provided with a starter file containing helper code.
- Complete the function implementations only.
- **Do NOT:**
 - Change function names
 - Change function parameters
 - Change return types
 - Add extra arguments
 - Use any built-in optimizer
- All code must be vectorized using NumPy.

Part 3 : Training Loop (Gradient Descent) (25 Marks)

- Initialize weights randomly.
- Update parameters manually using gradients.
- Store loss at every epoch.
- Print loss every 10 epochs.
- Include a plot of loss vs epochs to demonstrate decreasing loss.

Part 4 : Experiments (15 Marks)

- Train with learning rates: 0.1, 0.01, 1
- Plot Loss vs Epochs (three separate plots)

Part 5 : Decision Boundary Plot (15 Marks)

- After training, plot the decision boundary using:

$$x_2 = -(w_1 * x_1 + b) / w_2$$
- Overlay dataset points to visualize separation.
- Clearly explain:
 - Which side predicts class 1
 - Which side predicts class 0

Part 6 : Model Evaluation (10 Marks)

Use the same dataset you trained on to generate predictions and compute training accuracy.

- Implement accuracy manually:

```
def compute_accuracy(y_true, y_pred):
```

```
# Do not use sklearn.metrics
```

```
# Compute accuracy using NumPy only
```

- Report final training accuracy.